TUTORIAL 5B - CONSTRUCTING A PHYSICAL BOOT LOADER

This is a comprehensive, from-scratch tutorial on creating a **Universal UEFI Bootloader**. It guides you from writing the C code to testing it safely in a Virtual Machine (QEMU), and finally installing it on a USB stick for real hardware.

By the end of this guide, you will have written a C program that runs directly on your computer's hardware (bare metal), scans all connected hard drives (NVMe, SSD, USB) for operating systems like Windows or Linux, and presents a menu to boot them.

Step 1: Install Prerequisites

On your Ubuntu machine, install the compiler, UEFI library, and simulation tools.

```
sudo apt update
sudo apt install build-essential gnu-efi mtools qemu-system-x86
```

Create a workspace:

```
mkdir MyBootloader
cd MyBootloader
```

Step 2: The Source Code (main.c)

Create a file named main.c. This code handles the screen output, disk scanning, and booting logic.

```
#include <efi.h>
#include <efilib.h>

#define MAX_ENTRIES 20
#define TIMEOUT_SECONDS 5

typedef struct {
    CHAR16 Path[256];
    CHAR16 Name[64];
    EFI_HANDLE DeviceHandle;
```

```
} BootEntry;

BootEntry Entries[MAX_ENTRIES];
UINTN EntryCount = 0;

// Helper: Check if a file exists
BOOLEAN FileExists(EFI_FILE_HANDLE RootDir, CHAR16 *Path) {
    EFI_STATUS Status;
    EFI_FILE_HANDLE FileHandle;
    Status = uefi_call_wrapper(RootDir->Open, 5, RootDir, &FileHandle, Path,
EFI_FILE_MODE_READ, 0);
    if (EFI_ERROR(Status)) return FALSE;
    uefi_call_wrapper(FileHandle->Close, 1, FileHandle);
    return TRUE;
}

// Helper: Load and Start an OS
EFI_STATUS LoadAndStart(EFI_HANDLE ImageHandle, EFI_HANDLE
TargetDeviceHandle, CHAR16* Path) {
    EFI_STATUS Status;
    EFI_DEVICE_PATH *DevicePath;
    EFI_HANDLE NewImageHandle;

    DevicePath = FileDevicePath(TargetDeviceHandle, Path);
    if (!DevicePath) return EFI_INVALID_PARAMETER;

    Status = uefi_call_wrapper(BS->LoadImage, 6, FALSE, ImageHandle, DevicePath,
NULL, 0, &NewImageHandle);
    if (EFI_ERROR(Status)) {
        Print(L"Error loading %s: %r\n", Path, Status);
        uefi_call_wrapper(BS->Stall, 1, 3000000);
        return Status;
    }

    uefi_call_wrapper(ST->ConOut->ClearScreen, 1, ST->ConOut);
    Status = uefi_call_wrapper(BS->StartImage, 3, NewImageHandle, NULL, NULL);
    return Status;
}

// Scan a specific partition for OSs
```

```
void ScanFilesystem(EFI_HANDLE DeviceHandle, EFI_FILE_HANDLE RootDir) {
    EFI_STATUS Status;
    EFI_FILE_HANDLE EfiDir;
    EFI_FILE_INFO *FileInfo;
    UINTN FileInfoSize;
    UINT8 Buffer[1024];

    // 1. Check for Windows
    if (FileExists(RootDir, L"EFI\\Microsoft\\Boot\\bootmgfw.efi")) {
        StrCpy(Entries[EntryCount].Name, L"Windows Boot Manager");
        StrCpy(Entries[EntryCount].Path, L"\\EFI\\Microsoft\\Boot\\bootmgfw.efi");
        Entries[EntryCount].DeviceHandle = DeviceHandle;
        EntryCount++;
    }

    // 2. Check for Linux Distros
    Status = uefi_call_wrapper(RootDir->Open, 5, RootDir, &EfiDir, L"EFI",
EFI_FILE_MODE_READ, 0);
    if (EFI_ERROR(Status)) return;

    while (EntryCount < MAX_ENTRIES) {
        FileInfoSize = sizeof(Buffer);
        FileInfo = (EFI_FILE_INFO *)Buffer;
        Status = uefi_call_wrapper(EfiDir->Read, 3, EfiDir, &FileInfoSize, FileInfo);

        if (Status == EFI_NOT_FOUND || FileInfoSize == 0) break;
        if (StrCmp(FileInfo->FileName, L".") == 0 || StrCmp(FileInfo->FileName, L"..") == 0
|| StrCmp(FileInfo->FileName, L"Microsoft") == 0) continue;

        if (FileInfo->Attribute & EFI_FILE_DIRECTORY) {
            CHAR16 ProbePath[256];
            StrCpy(ProbePath, L"EFI\\");
            StrCat(ProbePath, FileInfo->FileName);
            StrCat(ProbePath, L"\\grubx64.efi");

            if (FileExists(RootDir, ProbePath)) {
                StrCpy(Entries[EntryCount].Name, FileInfo->FileName);
                StrCpy(Entries[EntryCount].Path, L"\\");
                StrCat(Entries[EntryCount].Path, ProbePath);
                Entries[EntryCount].DeviceHandle = DeviceHandle;
```

```
            EntryCount++;
        }
    }
  }
  uefi_call_wrapper(EfiDir->Close, 1, EfiDir);
}

EFI_STATUS EFIAPI efi_main (EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE
*SystemTable) {
  EFI_STATUS Status;
  EFI_INPUT_KEY Key;
  UINTN HandleCount;
  EFI_HANDLE *AllHandles;
  EFI_SIMPLE_FILE_SYSTEM_PROTOCOL *FileSystem;
  EFI_FILE_HANDLE RootDir;

  InitializeLib(ImageHandle, SystemTable);
  uefi_call_wrapper(ST->ConOut->ClearScreen, 1, ST->ConOut);
  uefi_call_wrapper(ST->ConOut->SetAttribute, 2, ST->ConOut, EFI_WHITE |
EFI_BACKGROUND_BLUE);

  Print(L"Scanning system...\n");

  // Scan all disks
  Status = uefi_call_wrapper(BS->LocateHandleBuffer, 5, ByProtocol,
&FileSystemProtocol, NULL, &HandleCount, &AllHandles);
  if (Status == EFI_SUCCESS) {
    for (UINTN i = 0; i < HandleCount; i++) {
      Status = uefi_call_wrapper(BS->HandleProtocol, 3, AllHandles[i],
&FileSystemProtocol, (void **)&FileSystem);
      if (!EFI_ERROR(Status)) {
        Status = uefi_call_wrapper(FileSystem->OpenVolume, 2, FileSystem,
&RootDir);
        if (!EFI_ERROR(Status)) {
          ScanFilesystem(AllHandles[i], RootDir);
          uefi_call_wrapper(RootDir->Close, 1, RootDir);
        }
      }
    }
    uefi_call_wrapper(BS->FreePool, 1, AllHandles);
```

```
    }

    // Timeout Loop
    if (EntryCount > 0) {
        Print(L"\nDefault boot: %s\n", Entries[0].Name);
        for (int i = TIMEOUT_SECONDS; i > 0; i--) {
            Print(L"Booting in %d seconds... (Press any key to stop)\r", i);
            for (int j = 0; j < 20; j++) {
                uefi_call_wrapper(BS->Stall, 1, 50000);
                Status = uefi_call_wrapper(BS->CheckEvent, 1, ST->ConIn->WaitForKey);
                if (Status == EFI_SUCCESS) goto manual_menu;
            }
        }
        LoadAndStart(ImageHandle, Entries[0].DeviceHandle, Entries[0].Path);
    } else {
        Print(L"No OS found. Press key to refresh...\n");
        uefi_call_wrapper(BS->WaitForEvent, 3, 1, &ST->ConIn->WaitForKey, NULL);
    }

manual_menu:
    uefi_call_wrapper(ST->ConIn->ReadKeyStroke, 2, ST->ConIn, &Key);

    while (1) {
        uefi_call_wrapper(ST->ConOut->ClearScreen, 1, ST->ConOut);
        Print(L"   UNIVERSAL BOOTLOADER   \n");
        Print(L"==========================\n\n");
        for (UINTN i = 0; i < EntryCount; i++) {
            Print(L"[%d] Boot %s\n", i + 1, Entries[i].Name);
        }
        uefi_call_wrapper(BS->WaitForEvent, 3, 1, &ST->ConIn->WaitForKey, NULL);
        uefi_call_wrapper(ST->ConIn->ReadKeyStroke, 2, ST->ConIn, &Key);

        if (Key.UnicodeChar >= '1' && Key.UnicodeChar <= '9') {
            UINTN Selection = Key.UnicodeChar - '1';
            if (Selection < EntryCount) {
                LoadAndStart(ImageHandle, Entries[Selection].DeviceHandle,
Entries[Selection].Path);
            }
        }
    }
```

```
    return EFI_SUCCESS;
}
```

Step 3: The Makefile

Create a Makefile to compile the code. **Important:** The indented lines (like gcc... or rm...) must start with a **TAB**, not spaces.

Makefile

```
ARCH            = x86_64
OBJS            = main.o
TARGET          = main.efi

EFIINC          = /usr/include/efi
EFIINCS         = -I$(EFIINC) -I$(EFIINC)/$(ARCH) -I$(EFIINC)/protocol
LIB             = /usr/lib
EFILIB          = /usr/lib
EFI_CRT_OBJS    = $(EFILIB)/crt0-efi-$(ARCH).o
EFI_LDS         = $(EFILIB)/elf_$(ARCH)_efi.lds

CFLAGS          = $(EFIINCS) -fno-stack-protector -fpic \
                -fshort-wchar -mno-red-zone -Wall
LDFLAGS         = -nostdlib -znocombreloc -T $(EFI_LDS) -shared \
                -Bsymbolic -L $(EFILIB) -L $(LIB) $(EFI_CRT_OBJS)

all: $(TARGET)

%.o: %.c
        gcc $(CFLAGS) -c $< -o $@

main.so: $(OBJS)
        ld $(LDFLAGS) $(OBJS) -o main.so -lefi -lgnuefi

%.efi: %.so
        objcopy -j .text -j .sdata -j .data -j .dynamic \
                -j .dynsym  -j .rel -j .rela -j .reloc \
                --target=efi-app-$(ARCH) $^ $@
```

```
clean:
	rm -f *.o *.so *.efi
```

Run make in your terminal. You should get a **main.efi** file.

Step 4: Booting on QEMU (Simulation)

This allows you to see if your menu works without restarting your computer.

1. Prepare Firmware We need the UEFI firmware files (OVMF). On Ubuntu 24.04, they are split into "Code" and "Vars". Copy the "Vars" file to your folder so we have write permissions.

```
cp /usr/share/OVMF/OVMF_VARS_4M.fd OVMF_VARS.fd
```

2. Create a Virtual Hard Disk We create a file named disk.img that acts as a 64MB USB stick.

```
# Create empty file
dd if=/dev/zero of=disk.img bs=1M count=64

# Format as FAT32
mformat -i disk.img -F ::
```

3. Install Bootloader & Fake OSs into the Image We will populate the virtual disk with folders so your scanner has something to find.

```
# Create UEFI structure
mmd -i disk.img ::/EFI
mmd -i disk.img ::/EFI/BOOT

# Copy our bootloader as the default boot file
mcopy -i disk.img main.efi ::/EFI/BOOT/BOOTX64.EFI
```

```
# Create a FAKE Windows (using our own file just to test detection)
mmd -i disk.img ::/EFI/Microsoft
mmd -i disk.img ::/EFI/Microsoft/Boot
mcopy -i disk.img main.efi ::/EFI/Microsoft/Boot/bootmgfw.efi

# Create a FAKE Ubuntu
mmd -i disk.img ::/EFI/ubuntu
mcopy -i disk.img main.efi ::/EFI/ubuntu/grubx64.efi
```

4. Run QEMU Run this command to boot your virtual machine:

```
qemu-system-x86_64 \
   -drive if=pflash,format=raw,readonly=on,file=/usr/share/OVMF/OVMF_CODE_4M.fd \
   -drive if=pflash,format=raw,file=OVMF_VARS.fd \
   -drive format=raw,file=disk.img \
   -net none
```

What to expect: You should see a countdown, and then a menu listing "Windows Boot Manager" and "ubuntu".

Step 5: Booting on Real Hardware

Now we put it on a real USB stick.

1. Format USB Stick

- Plug in your USB.
- Find its device name (e.g., /dev/sdb) using lsblk.
- Unmount it: sudo umount /dev/sdb1
- **Format as FAT32:**

```
sudo mkfs.vfat -F 32 -n "MYBOOT" /dev/sdb1
```

(Be careful! Double-check the device name /dev/sdb1 is correct).

2. Copy the File

- Mount the USB:

```
sudo mkdir -p /mnt/usb
sudo mount /dev/sdb1 /mnt/usb
```

- Create the folders:

```
sudo mkdir -p /mnt/usb/EFI/BOOT
```

- Copy your main.efi and **rename** it to BOOTX64.EFI:

```
sudo cp main.efi /mnt/usb/EFI/BOOT/BOOTX64.EFI
```

- Unmount safely:

```
sudo umount /mnt/usb
```

3. Boot It

- Shutdown your PC.
- Insert the USB.
- Turn on and press **F12,** Del or F2 (usually **F12** for Dell/Lenovo, **F9** for HP, **F8** for Asus) to enter the Boot Menu.
- Select **"UEFI: MYBOOT"**.
- **Note:** If it fails, disable "Secure Boot" in your BIOS settings.

Your custom bootloader will now run, scan your actual hard drives, and let you boot your real Windows or Linux installations.

Exercise 1: Identification banner

Show your full name and student ID in a header on every screen: the scanning screen, the countdown and the menu.

- Write one function DrawHeader() and call it after every ClearScreen.
- Use SetAttribute to give the header its own colours (for example yellow on blue) and restore the normal colours afterwards.
- Centre the text: query the screen size with ST->ConOut->QueryMode(ConOut, ConOut->Mode->Mode, &Cols, &Rows) and use SetCursorPosition. Real firmware text modes range from 80×25 to well over 200 columns, so never hard-code the width.
- Install on the USB stick (Step 5) and boot it through the one-time boot menu.
- **Check:** a photo of the physical screen showing your name and ID. If you can, boot a second, different machine and report the Cols × Rows each one gives.

Exercise 2: Arrow-key menu with highlight

Replace the "press 1–9" menu with a menu you navigate with the arrow keys.

- Up/Down move the selection (Key.ScanCode == SCAN_UP / SCAN_DOWN); Enter boots it (Key.UnicodeChar == CHAR_CARRIAGE_RETURN).
- Draw the selected line in inverted colours with SetAttribute.
- Home/End jump to the first and last entry; the selection wraps around.
- Redraw only the two lines that change, not the whole screen, using SetCursorPosition.
- Keep number keys as shortcuts, and make all 20 entries reachable.
- Next to each entry, show which disk it is on with DevicePathToStr(DevicePathFromHandle(Handle)) (free the string with FreePool), so you can tell the internal NVMe/SATA disk from the USB stick.
- To test more than 9 entries, add fake folders **on the USB stick only**, for example /mnt/usb/EFI/test01 … test15, each holding a copy of main.efi named grubx64.efi.
- **Check:** a short phone video on a physical machine with at least one installed OS: navigate the list, then select the real Windows or Ubuntu entry and show it actually starts. In your report, explain which part of each device path tells you NVMe, SATA or USB.